

100 Days of Azure DevOps

Day 15 Handout — Rebase rewrites history. Merge keeps the plot twists. Know who is watching the timeline.

2 - Azure Repos & Git Mastery · Personal lab only · Educational content · Not a sales pitch

Architecture A — integrating feature/* onto main (same files, different graph)

Operation	Graph result	Safe on
Merge commit	New commit with two parents	Shared branches if policy allows
Squash merge (ADO PR)	One new commit on main; feature SHAs dropped	feature/* → main (this lab default)
Rebase onto main	Replay your commits; new SHAs	Local feature/* only, then force-with-lease
Fast-forward	Pointer move, no extra commit	When feature is linear on main
Cherry-pick	Copy a commit SHA onto another branch	Hotfix one commit, not a whole feature
Soft reset	Move branch, keep index/worktree	Rewrite last local commit before push
Hard reset	Move branch, destroy uncommitted (and matching files)	Never on shared main

Architecture B — decide rebase vs merge vs squash

Situation	Choose	Do not
Unpushed local commits, main moved	git fetch; git rebase origin/main	rebase origin/main after teammates pulled your branch
PR into main in this lab	Squash (ADR)	Merge commit unless you need the DAG
Public main / already shipped SHA	Revert or forward fix	rebase -i of origin/main
One commit actually belongs elsewhere	cherry-pick <sha>	merge the whole unrelated branch
Conflict during rebase	edit, git add, git rebase --continue	git rebase --abort only after you understand the conflict
Need to update a remote feature after rebase	git push --force-with-lease	git push --force (overwrites others)

Architecture C — pitfalls that delete someone else's Thursday

Pitfall	What you see	Fix
Rebase shared main	Everyone's PRs cannot FF; CI SHAs vanish	Never rebase main; revert forward
git push --force	Teammate's commit disappeared from the branch	--force-with-lease or new branch
Conflict markers committed	<<<<<<< HEAD in source	Resolve; git add; continue; grep for <<<<<<<
Cherry-pick a merge commit blindly	Wrong tree	cherry-pick -m 1 only if you mean it
Reset --hard after stashless work	File gone from disk	reflog: git reflog / git reset --hard ORIG_HEAD quickly
Squash + rebase both 'to be clean'	Unrecognizable history twice	Pick one strategy per PR

One-liner to remember

Rebase rewrites history; merge preserves the plot twists — choose based on whether anyone else is watching that timeline.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal repo only. Manufacture a conflict, resolve it, rebase a feature onto main. Do not force-push main.

- 1 `git switch main` && `git pull`. Create `feature/day15-a` and `feature/day15-b` from the same SHA.
- 2 On `day15-a`: edit `docs/day15-conflict.md` line 1 to 'alpha'. Commit. Push.
- 3 On `day15-b`: edit the same file line 1 to 'bravo'. Commit.
- 4 `git fetch origin` && `git rebase origin/main` (if main moved) then `rebase/merge day15-a` into `day15-b` until conflict appears.
- 5 Resolve conflict in the editor. `git add docs/day15-conflict.md` && `git rebase --continue` (or `git commit` if you used merge).
- 6 If you rebased a branch already on origin: `git push --force-with-lease`. Never push `--force` to main.
- 7 Write 5 lines in `docs/day15-when.md`: when you choose merge commit vs squash vs rebase. Commit on the feature branch.

Rebase a feature onto main, then push with lease

```
git fetch origin
git switch feature/day15-b
git rebase origin/main
# fix conflicts → git add docs/day15-conflict.md → git rebase --continue
# abort if lost: git rebase --abort
git push --force-with-lease origin feature/day15-b
```

Done checklist

- Can explain rebase vs merge vs squash with the parent graph
- Resolved a real conflict (markers gone)
- Used `--force-with-lease` on a feature branch, never on main
- Wrote when-to-choose notes in the repo

Tomorrow — Day 16

Git hooks & pre-commit